Earlier you learned that constraints can be NOT VALID. This is a way to enforce the constraint for new row modifications, but not for existing rows that aren't being checked.

Constraint enforcement can also be deferred which is slightly different. What does that mean?

Deferring Constraint Checks

PostgreSQL supports deferring constraint checks within a transaction. UNIQUE, PRIMARY KEY, Foreign Key (REFERENCES), and Exclusion (EXCLUDE) constraints are *deferrable* although they are all checked immediately by default. This means that if you wish to make the constraint deferrable, you'll need to create it that way from the beginning or drop it and create it again with the DEFERRABLE option enabled.

Foreign Key and Exclusion constraints can have their deferrable behavior modified using the SET CONSTRAINT[8] operation within a transaction.

Where can this be used? In the Hashrocket post *Deferring Constraints in PostgreSQL*,[9] the author walks through a use case for deferring constraints in a Rails app.

Items in a list have unique position values. To enforce unique position values a UNIQUE constraint is used on the position column. Without deferring the constraint enforcement, successive UPDATE calls would duplicate positions as list items are re-ordered, which would violate the Unique constraint definition.

To work around this, UPDATEs must be made within a transaction, and the Unique checks are delayed until the transaction commits. This mean it's possible to create duplicate position values, basically ignoring the Unique constraint, within the scope of the transaction before it's committed. They do need to all be unique by the time of commit, though.

Once all the positions are swapped around and unique again, the transaction is committed and constraint enforcement happens at that time.

By defining the constraint as DEFERRABLE INITIALLY DEFERRED it's possible to change the default behavior from being checked immediately to being checked at the end in a transaction.

Using that option would make the behavior described above the default behavior. If you realize later that this option would be better as the default

8. https://www.postgresql.org/docs/current/sql-set-constraints.html
9. https://hashrocket.com/blog/posts/deferring-database-constraints